



HACKFEST 2026

Problem Statement Documentation & Technical Submission

(Template / Dummy Format – for Team Reference)

Team ID	KH097
Team Name	Tech4
Team Leader Name	Zaara Shaikh
Problem Statement No.	PS16
Problem Statement Name	Autonomous Scheme-Bundle optimizer for citizens
Date of Submission	11 Sept 2026



SECTION 1 — TEAM DETAILS

Team ID **KH097**

Team Name **Tech4**

Team Leader Name **Zaara Shaikh**

Team Members Name(s)

- **Pritika Warad**
- **Samruddhi Shelke**
- **Asawari Natave**

Problem Statement Number **PS16**

Problem Statement Name **Autonomous Scheme-Bundle optimizer for citizens**

SECTION 2 — UNDERSTANDING OF THE PROBLEM

Problem Statement

Citizens are often eligible for multiple government schemes at once, but existing scheme-discovery tools only tell them which individual schemes they may qualify for. They don't tell citizens which of those schemes can actually be combined, which conflict with or exclude one another, or which combination gives them the most overall benefit. PS16 asks for an agentic assistant that takes a citizen's profile, finds eligible schemes, detects conflicts between them, and recommends an optimized combination.

Problem Analysis / Background

Government scheme eligibility criteria are published as free-text prose spread across many sources, not as clean structured data — so even automated matching is non-trivial. On top of that, two individually eligible schemes are not always jointly compatible: some are mutually exclusive, some depend on others, and benefit types (cash, subsidy, insurance, training) aren't directly comparable. National platforms such as myScheme (4,000+ schemes) and UMANG solve discovery well, but leave this harder combination problem entirely to the citizen. This affects any citizen trying to make informed use of the schemes they're entitled to, and any assisted-service worker helping them do so.

Proposed Solution

MaxLabh is an agentic assistant that takes a citizen profile, matches it against a curated set of central government schemes, flags conflicts/incompatibilities between eligible schemes, and recommends an optimized, non-conflicting bundle with a plain-language explanation and an application-readiness checklist. Its core design choice is to keep the language model out of the actual eligibility and conflict decisions — a language model helps extract structured rules from free-text scheme criteria and explain results in plain language, but rule-matching and conflict logic run as plain deterministic code. This keeps recommendations traceable back to a specific rule rather than resting on a generated claim.

SECTION 3 — RESEARCH

Research Conducted

We reviewed India's national scheme-discovery platforms (myScheme, UMANG) and international benefits-screening tools (BenefitsCheckUp, mRelief) to establish what already exists and where the actual gap for PS16 lies. We also identified the Indian Government Welfare Schemes dataset (Kaggle, 4,702 records) as a starting source for scheme eligibility text, to be reviewed and narrowed to a workable subset for the prototype rather than used wholesale.

Existing Solutions / Similar Work

Platform	Standout Feature	Gap vs. Our Objective
myScheme (Govt. of India)	4,000+ schemes, personalized eligibility questionnaire, AI chatbot	Checks eligibility per scheme; no cross-scheme conflict detection or bundle optimization
UMANG (Govt. of India)	Unified access to 1,984+ services across departments	Service directory, not a reasoning/recommendation engine
BenefitsCheckUp (NCOA, USA)	Anonymous screening, 2,000+ programs, personalized report	Returns a list of eligible programs, not an optimized combination
mRelief (Chicago, USA)	Low-friction SMS-based screening	Narrow scope (12 programs), no optimization layer

Across all four, the common stopping point is eligibility discovery. None model conflicts between schemes or compute an optimal combination — that gap is what MaxLabh targets.

References / Sources

- myScheme — <https://www.myscheme.gov.in/>
- UMANG — <https://web.umang.gov.in/>
- BenefitsCheckUp — <https://benefitscheckup.org/>
- mRelief — <https://www.mrelief.com/>
- Indian Government Welfare Schemes dataset, 4,702 records (Kaggle: premsargara/indian-government-beneficiary-schemes) — <https://www.kaggle.com/datasets/premsargara/indian-government-beneficiary-schemes>

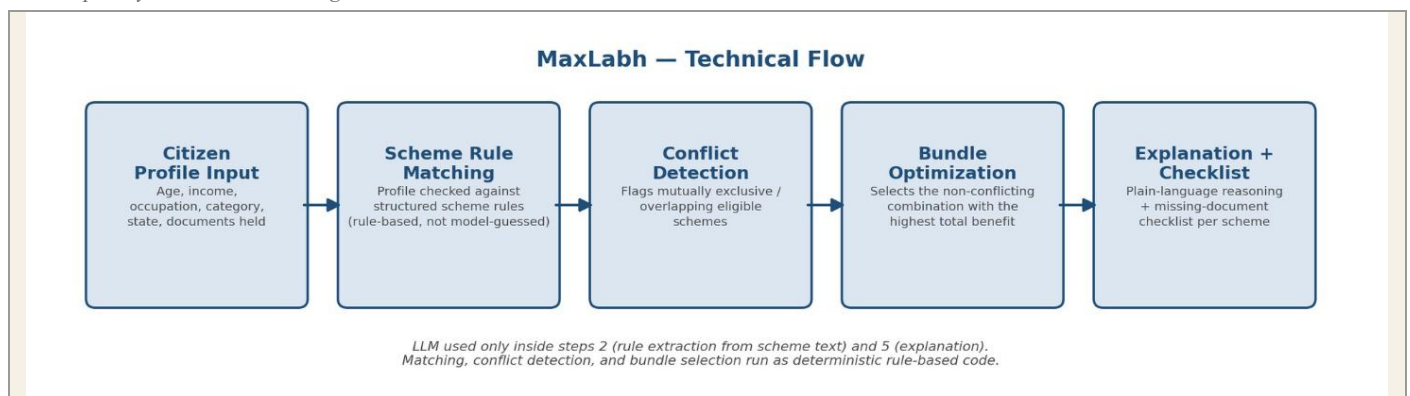
SECTION 4 — TECHNICAL DOCUMENTATION

Technology Stack

Reference stack — may be adjusted during the build as needed: Python, LangChain (LLM orchestration for rule extraction and explanation only), Streamlit (citizen-facing UI), Pandas (data handling), a constraint-based bundle selector (greedy heuristic, upgraded to a simple solver library if time permits). Scheme source: Indian Government Welfare Schemes dataset (Kaggle, 4,702 records), narrowed to a reviewed subset of central schemes.

Technical Flow / System Architecture Diagram

Attach / paste your architecture diagram below.



Working / Implementation Approach

The citizen fills a short profile (age, income, occupation, category, state, documents held). The system matches this profile against a set of scheme rules — where scheme eligibility text is unstructured, an LLM-assisted step helps convert it into a structured, checkable rule, which the team reviews before use. Matching and conflict-checking run as plain rule-based logic, not model inference, so results stay traceable. Eligible-but-conflicting schemes are identified, and the optimizer selects the combination that maximizes benefit without violating a conflict. The output is the selected bundle, a plain-language explanation of why each scheme was included or excluded, and a checklist of documents still needed.

AI/ML Details (if applicable)

Dataset: Indian Government Welfare Schemes dataset (Kaggle, 4,702 records), narrowed to a reviewed working subset of central schemes with free-text eligibility criteria. LLM use is intentionally bounded to two tasks — extracting candidate structured rules from scheme text, and generating a plain-language explanation from the system's own decision output. The LLM does not itself decide eligibility, conflicts, or the final bundle — those run as deterministic code so results can be explained and re-checked rather than taken on trust.

GitHub Repository Link

https://github.com/zaara01-sk/KH097_Tech4.git

SECTION 5 — ORIGINALITY & PLAGIARISM DECLARATION

☐ I confirm that the solution submitted by our team is developed during the hackathon. We have not submitted a pre-built project as our primary solution. Any open-source code, APIs, datasets, libraries, templates, or AI-assisted development used by us have been appropriately acknowledged.

Open-Source / External Resources Used

- LangChain, Streamlit, Pandas — open-source libraries
- API — Gemini/groq api
- Indian Government Welfare Schemes dataset, 4,702 records (Kaggle: premsargara/indian-government-beneficiary-schemes) — used with attribution, reviewed and narrowed before use, not treated as an authoritative legal source
- Kurukshetra 2.0 documentation template — provided by organisers

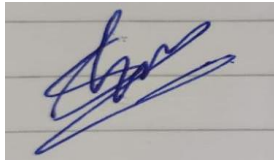
AI Tools Used

Claude (Anthropic) and Gemini — used for pre-event research on existing scheme-discovery platforms, architecture planning, documentation drafting . All work/project during the 24-hour build is the team's own efforts, and every team member can explain any part of it on request.

SECTION 6 — UPLOADS / ATTACHMENTS CHECKLIST

- ☐ Technical Flow / Architecture Diagram
- ☐ Research / Reference Document (if applicable)
- ☐ Project Documentation / Technical Report
- ☐ GitHub Repository Link
- ☐ Screenshots of Working Prototype
- ☐ Dataset / Sample Data (if applicable)
- ☐ Additional Diagrams (if any)

Team Leader Signature: _____



Date: 11 sep 2026 _____